

Prueba Técnica Full Stack - Ecosistema Instalación (Quarkus)

1. Descripción General

Este proyecto consiste en la construcción de un mini-ecosistema de evaluación de créditos compuesto por:

- **Frontend (UI)**
- **Microservicio Orquestador (ms-orchestrator)**
- **Microservicio Externo de Riesgos (ms-risk)**

Todo el backend está desarrollado en **Java con Quarkus**, siguiendo una arquitectura de microservicios desacoplados.

2. Arquitectura del Sistema

Flujo general:

1. El usuario ingresa datos en el Frontend.
 2. El Frontend envía la solicitud al **ms-orchestrator**.
 3. El Orquestador valida y envía la información al **ms-risk**.
 4. El ms-risk calcula el nivel de riesgo del cliente.
 5. El ms-orchestrator consolida la respuesta y la devuelve al Frontend.
-

3. Tecnologías y Versiones Recomendadas

Backend (Quarkus Microservices)

- Java: **21**
- Quarkus: **3.8+ (o última estable 3.x)**
- Maven: **3.9+**
- REST Client (Quarkus Rest Client Reactive)
- Jackson JSON
- SmallRye OpenAPI

Frontend

- Node.js: **20 LTS**

- Framework: **Vue 3 o React 18**
- Vite (recomendado)
- Axios

Infraestructura

- Docker: **latest stable**
 - Docker Compose
-

4. Microservicio: ms-risk (Externo de Riesgo)

Responsabilidad

Este microservicio evalúa el nivel de riesgo del cliente basado en datos como:

- Edad
- Ingresos
- Historial crediticio

Endpoint principal

POST /risk/analyze

Request ejemplo

```
{
  "age": 30,
  "income": 2500,
  "creditHistory": "good"
}
```

Response ejemplo

```
{
  "riskLevel": "LOW",
  "score": 85
}
```

Lógica básica

- Ingresos altos + buen historial = riesgo bajo
 - Ingresos bajos o historial malo = riesgo alto
-

5. Microservicio: ms-orchestrator

Responsabilidad

- Recibir solicitudes del frontend
- Validar datos
- Consumir ms-risk
- Retornar respuesta consolidada

Endpoint principal

POST /credit/evaluate

Request ejemplo

```
{  
  "name": "Moises",  
  "age": 27,  
  "income": 3000,  
  "creditHistory": "good"  
}
```

Response ejemplo

```
{  
  "approved": true,  
  "riskLevel": "LOW",  
  "message": "Credit approved"  
}
```

Comunicación interna

- Usa REST Client de Quarkus para consumir ms-risk
-

6. Frontend (UI)

Funcionalidades

- Formulario de evaluación de crédito
- Envío de datos al backend
- Visualización de resultado

Flujo

1. Usuario llena formulario
2. Click en "Evaluar crédito"
3. Se muestra resultado:
 - Aprobado / Rechazado
 - Nivel de riesgo

Endpoint consumido

POST `http://localhost:8080/credit/evaluate`

7. Docker

`docker-compose.yml` (estructura base)

version: '3.8'

services:

ms-risk:

build: ./ms-risk

ports:

- "8081:8080"

ms-orchestrator:

build: ./ms-orchestrator

ports:

- "8080:8080"

depends_on:

- ms-risk

frontend:

build: ./frontend

ports:

- "3000:3000"

8. Git Ignore (Completo y resumido)

Java / Quarkus

target/

*.log

*.class

*.jar

*.war

Maven

```
.mvn/  
!mvn/wrapper/maven-wrapper.jar
```

```
# IDE  
.idea/  
*.iml  
.vscode/
```

```
# OS  
.DS_Store  
Thumbs.db
```

```
# Environment  
.env
```

```
# Node (Frontend)  
node_modules/  
dist/  
build/
```

```
# Docker  
*.pid
```

```
# Quarkus dev mode  
.quarkus/
```

9. Instalación (Solo instalación)

9.1 Clonar el repositorio

```
git clone https://github.com/tu-usuario/credit-system.git  
cd credit-system
```

9.2 Requisitos previos

- Java 21
- Maven 3.9+
- Node.js 20 LTS
- Git
- Docker (opcional)

Verificación:

```
java -version
```

```
mvn -version  
node -v  
git --version  
docker -v
```

9.3 Instalación ms-risk

```
cd ms-risk  
mvn clean install
```

9.4 Instalación ms-orchestrator

```
cd ms-orchestrator  
mvn clean install
```

9.5 Instalación frontend

```
cd frontend  
npm install
```

9.6 Construcción con Docker (opcional)

Desde la raíz del proyecto:

```
docker-compose build
```

10. Buenas Prácticas Aplicadas Buenas Prácticas Aplicadas Buenas Prácticas Aplicadas

- Arquitectura de microservicios
 - Separación de responsabilidades
 - Comunicación REST
 - Uso de DTOs
 - Manejo de dependencias con Maven
 - Configuración por entorno
 - Contenerización con Docker
-

11. Posibles mejoras

- Autenticación JWT
- Base de datos PostgreSQL
- Circuit Breaker (SmallRye Fault Tolerance)
- Logging centralizado
- Tests unitarios con JUnit + Mockito